Hand-off: Marine Vessel 3D Asset Pipeline

Oct 15, 2026 · @Douglas Xu

Where things stand

The plan and tech stack are decided on paper; nothing is built yet. The next step is a spike on one system (propulsion), end to end. There is no CAD source yet, so the assets are synthetic and the CAD integration is simulated.

- **Goal:** lightweight, three.js-renderable marine vessel assets (hull, machinery, internal structure, outfitting) that explode into subsystems, equipment and components and collapse back, carry physical properties tied to function, are OpenUSD-compatible, and connect to backend APIs.
- **Constraints:** open source only, container-based, deployable on a laptop, on-prem or cloud. Viewer audience (internal engineers or external clients) is still open.
- **Done:** plan, OCX assessment, tech stack, roadmap. **Not done:** any code, schema file, generator, converter, viewer or backend.

Decisions already made

Topic	Decision	Why
Source of truth	OpenUSD, in git	Composition, variants and relationships; everything else rebuilds from it
Web format	glTF/GLB, built from USD	three.js USD support is partial
Live data	Backend, fetched by ID	Keeps files static and cacheable
Join key	One permanent UUID per part (`marine:id`)	Same ID in USD, GLB, graph and API
OCX	Hull structure only	Not designed for machinery, piping or outfitting
Breakdown and codes	VIS/Gmod (SFI as fallback), plus ISO 19848 names for live data	Human-readable ID beside the UUID

Topic	Decision	Why
CAD exchange	STEP AP242 via OpenCascade	The realistic format real CAD will hand over
Authoring	Code-first generators (build123d or CadQuery)	Reviewable, repeatable, headless; Blender and FreeCAD in supporting roles
CAD simulation	Generators also emit STEP; a separate ingest step reads it into USD	Exercises the real-CAD path from day one
Explode	Assembled pose is canonical; direction and distance stored per child	No baked exploded geometry
Schema	Codeless `marine:` API schema first	Portable to third-party USD tools

Conventions to follow from day one

- **Hierarchy:** Vessel, Zone or Deck, System, Equipment, Component. USD `kind`: `assembly` for vessel, zone and system; `component` for equipment; `subcomponent` for parts.
- **Units and axes:** `metersPerUnit = 1`, `upAxis = Z`. Ship frame: origin at aft perpendicular, centerline and baseline; x forward, y to port, z up. The viewer converts to Y-up once, at its root.
- **IDs:** `marine:id` (UUID) is permanent and never reused. `marine:code` holds the VIS/Gmod path. glTF node names are the UUIDs.
- **Attributes:** `marine:id`, `code`, `class`, `function`, `material`, `mass`, `diameter`, `length`, `thickness`, `explodeDir`, `explodeDist`, `lodLevel`. Dimensions are generator parameters, never typed twice. Position comes from the transform.
- **Connectivity:** USD relationships `marine:connectedTo` and `marine:serves`, mirrored into the graph store by the build.
- **LOD:** L0 proxy per assembly, L1 equipment, L2 full components with instanced fasteners. Triangle budgets are starting guesses until profiled.
- **Builds:** pinned container, deterministic outputs, content-hashed file names, released versions immutable.

First tasks: the propulsion spike

The spike is roughly three weeks (a rough guess, to be reset by its results). It is done when the gate passes: explode works, and IDs match across USD, GLB and the API.

- [ ] Read the current OCX 3.x schema documentation; confirm or correct the structural boundary
- [ ] Verify licences and versions: VIS/Vista, OCX, OpenUSD, build123d, S3-compatible storage choice
- [ ] Run the one-week bake-off: a controllable-pitch propeller, a centrifugal pump and a hull midship section built with build123d, Blender via Claude, and FreeCAD via Claude; judge on time to a validated asset, dimensional accuracy and fixes needed
- [ ] Write the codeless `marine:` schema and the monorepo skeleton (`generators/` , `assets/` , `library/` , `schema/` , `build/` , `viewer/`)
- [ ] Build propulsion generators (shaft line, gearbox, engine, propeller with hub, blades, cone and instanced bolts) emitting USD, GLB and STEP from one parameter set
- [ ] Build the STEP ingest adapter that reads the generated STEP into USD, as it would read real CAD
- [ ] Write the USD to GLB converter (IDs as node names, properties in `extras`) with round-trip tests
- [ ] Generate default explode vectors and L0 proxies; add the explode and proxy validators
- [ ] Build the minimal viewer: manifest loading, tree, picking, explode slider per level
- [ ] Stand up one API call, `GET /assets/{id}` , joined by ID
- [ ] Profile on the slowest target device and set real triangle budgets
- [ ] Test the assets in at least one third-party USD viewer; test Blender's USD custom-property export on the pinned release

Assumptions to verify and open questions

Stated from memory, not checked:

- OCX 3.x scope (hull structure only), and the licences and versions of VIS/Vista, OCX, MinIO alternatives, build123d and OpenUSD.
- The standards claims around VIS/Gmod, ISO 19848, STEP AP215/216/218 and Asset Administration Shell.
- All triangle budgets and chunk sizes; the three-week spike estimate.

Open questions for the owner:

- Is there an existing breakdown or ID scheme to adopt, or is VIS/Gmod acceptable as the default?
- Is the viewer for internal engineers or external clients? This sets triangle budgets, hosting and access control.
- Do edits need to flow back to CAD, or is USD one-way?
- What are the slowest target devices and browsers?

Known risks: custom USD schemas are not portable by default; the USD to GLB converter is custom code and needs round-trip tests; Blender's USD export of custom properties varies by version.

Where the full plan lives

The full plan is `Marine Vessel 3D Asset Pipeline: Plan and Tech Stack`. Read these sections first:

1. OCX assessment and complementary standards, to see where OCX stops.
2. Asset model and Explode and collapse, for the schema and runtime behaviour.
3. Authoring pipeline, for the generator approach and the bake-off.
4. Backend, ontology and simulated CAD integration, for the ingest design.

The doc also holds the architecture and roadmap drawings, the validator list, the full tech stack table and the sources.